

1.2 Neurons, Activations, and Feedforward Nets

A weighted sum, a nonlinearity, and why stacking linear layers is not enough.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A transformer is a large neural net. Before attention, you need one unit, a nonlinearity, and a stack of layers. That is this note.

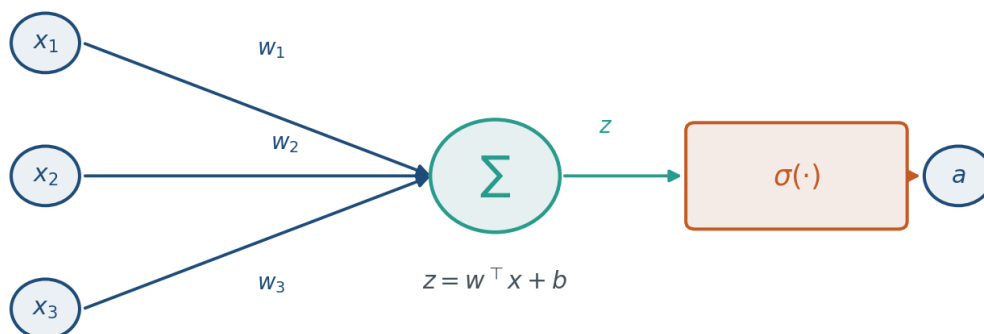
1. One neuron

A neuron takes a vector x , forms a weighted sum plus a bias, then applies a scalar nonlinearity σ :

$$z = w^\top x + b, \quad a = \sigma(z).$$

z is the **pre-activation**. a is the **activation**. w and b are the parameters you will train.

One neuron: weighted sum, then a nonlinearity



If σ is a step function, this is a 1950s perceptron. If σ is a sigmoid, it is logistic regression in one unit. Modern nets mostly use **ReLU** in hidden layers, **tanh** or GELU in some transformer blocks, and **softmax** on mutually exclusive class scores.

On the board, always write z and a as two boxes. Students who skip z cannot take derivatives next note.

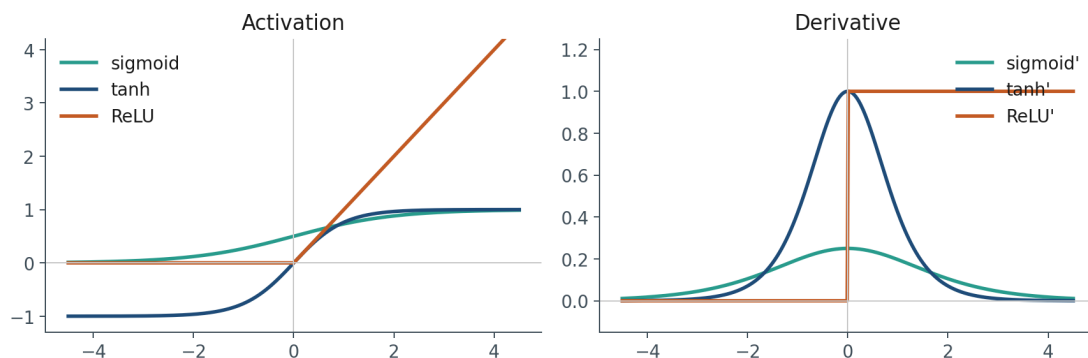
2. Why the nonlinearity is not optional

A layer without σ is an affine map. Two affine maps compose to one affine map:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Stack as many linear layers as you want: you still have a linear classifier. The bend is the point of the hidden layer.

Without a nonlinearity, stacked layers collapse to one linear map



ReLU is cheap and sparse (negative pre-activations become 0). Sigmoid saturates: its derivative is near 0 for large $|z|$, which will matter when we talk about vanishing gradients in Week 2.

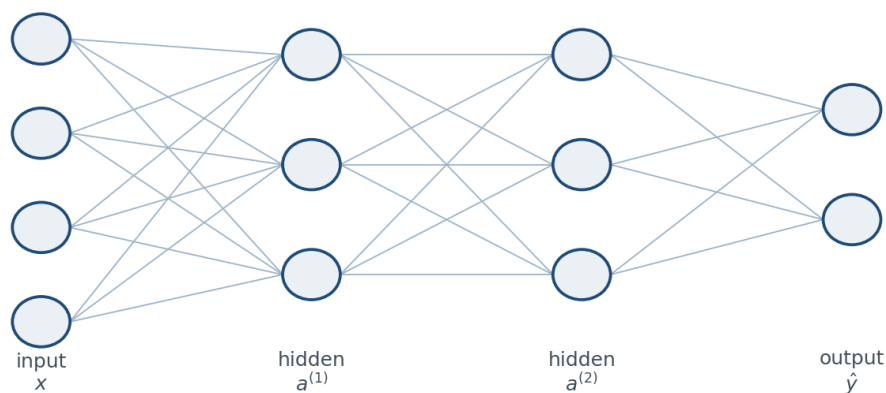
GELU is a smooth cousin of ReLU; you will see it in transformer MLPs. You do not need its formula today. You do need: **no bend** $\Rightarrow$ **no extra function class**.

3. A feedforward net

A **multi-layer perceptron (MLP)** is a stack of dense layers. Layer ℓ sees the previous activations, not the raw input (except $\ell = 1$):

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \sigma(z^{(\ell)}), \quad a^{(0)} = x.$$

A feedforward net: each layer is $a^\ell = \sigma(W^\ell a^{\ell-1} + b^\ell)$

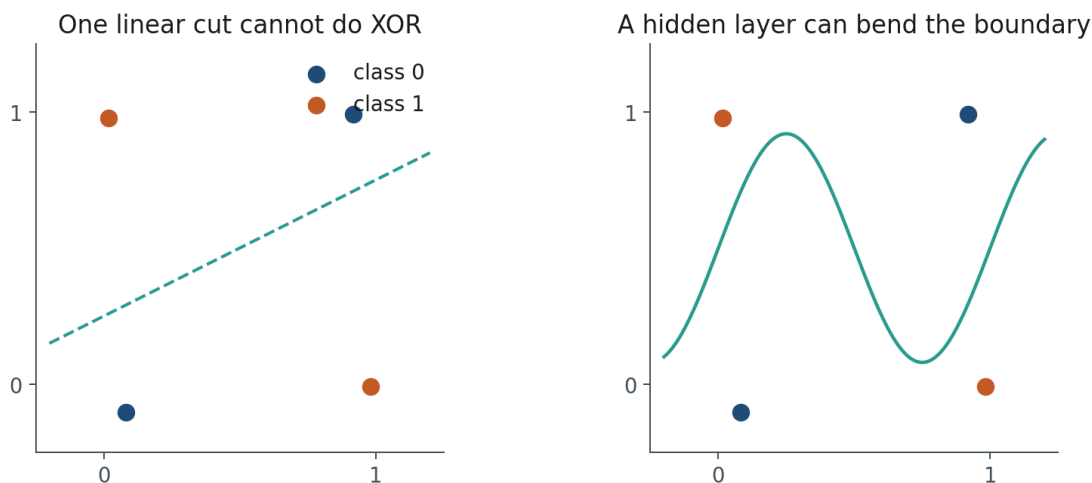


Every arrow in that picture is one entry of some $W^{(\ell)}$. A “fully connected” layer means every unit in layer $\ell - 1$ talks to every unit in layer ℓ .

A transformer block still contains an MLP; attention is extra mixing across **positions**. This note is the per-position map.

4. What a hidden layer buys you

A single linear unit cannot separate XOR: the positive class is not on one side of a line. Two hidden units, then a linear output, can fold the space so the classes become linearly separable.



That is the whole argument for depth, in miniature. Language models stack many such nonlinear maps (and, from Week 3, attention) so that “cat sat on the mat” and “the mat sat on the cat” do not look like the same vector.

Lab 1 trains this XOR net with autograd. Today you only need the forward pass on paper.

5. Teaching this note

30–40 minutes at the board. Derive one neuron, then the two-linear-layers collapse, then XOR as the reason for σ . Do the worked example with ReLU before you play video. Play **2:42–11:34** of 3Blue1Brown (neurons, layers, why layers, edge detectors). Leave ReLU vs. sigmoid at **17:03** if you have two extra minutes. Lab 1 is the studio follow-up, not this block.

6. Worked example

Take $x = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$, $w = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, $b = 1$.

$$z = 0.5 \cdot 1 + 0.5 \cdot (-2) + 1 = 0.5, \quad \text{ReLU}(z) = \max(0, 0.5) = 0.5.$$

Change the second coordinate to -4 : $z = 0.5 - 2 + 1 = -0.5$, so $\text{ReLU}(z) = 0$. The unit is off; its local derivative is 0.

Now drop σ and compose two layers in 1-D: $W_1 = 2$, $b_1 = 1$, $W_2 = 3$, $b_2 = 0$. The map is $3(2x + 1) = 6x + 3$, still a line. Write that product on the board before you draw XOR.

7. Where students get stuck

- Mixing up z and a , then trying to backprop through the wrong box.
 - Believing “more layers” automatically buy a richer model even with no σ .
 - Treating ReLU’s zero derivative as only a bug (it is also sparsity) or only a feature (dead ReLUs).
-

8. Video

Watch [3Blue1Brown: But what is a neural network?](#).

Pause at **2:42** (a neuron as a number), **5:31** (why layers), **8:38** (edge detection as a hidden feature), and **17:03** (ReLU vs. sigmoid). The digit-classifier story is the intuition; our algebra is z then σ .

9. Practice

1. Write $a^{(2)}$ for a two-hidden-layer net as a nested formula in $W^{(1)}$, $W^{(2)}$, $W^{(3)}$ and σ .
2. Drop every σ . What class of functions remains?
3. Why is ReLU’s derivative 0 on the negative half-line a feature *and* a failure mode?
4. For $x = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$, $w = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$, $b = 0.5$, compute z and $\text{ReLU}(z)$. Then compute $\sigma(z) = 1 / (1 + e^{-z})$ to two decimals. Which unit is closer to “off”?
5. Let $W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$, $W_2 = \begin{bmatrix} 3 & 1 \end{bmatrix}$, both biases zero, no σ . Write the composed map $W_2 W_1 x$ as a single row times x .